INFORME TÉCNICO Y DE EVIDENCIAS DE APRENDIZAJE: DESARROLLO, DESPLIEGUE Y PRUEBAS DE INTEGRACIÓN DE API RESTFUL

Proyecto: API RESTful para Sistema de Alquiler de Vehículos
Programa de Formación: Análisis y Desarrollo de Software (ADSO)
Centro de Formación: CTPI — SENA Regional Cauca
Instructor Asignado: Franco Reina
Aprendiz: Santiago Avirama Gurrute
Entorno y Despliegue: Node.js (ESM), Express, PostgreSQL (Neon.tech), Render Web Service, Thunder Client
Fecha de Presentación: 14 de Septiembre de 2026

---

1. Resumen y Alcance del Proyecto

El presente informe técnico documenta de manera formal y rigurosa el ciclo de vida completo de ingeniería de software correspondiente al diseño, codificación, configuración, persistencia, aseguramiento, despliegue continuo en la nube y pruebas de integración de una API RESTful de Alquiler de Autos.
La solución desarrollada conforma un núcleo backend desacoplado que centraliza el inventario vehicular, el control y registro de clientes mediante autenticación criptográfica sin estado (stateless) y la trazabilidad transaccional de los contratos de alquiler de vehículos, garantizando integridad referencial, seguridad perimetral y disponibilidad continua.

Matriz del Stack Tecnológico Implementado

Componente	Tecnología / Especificación	Propósito Técnico en la Solución
Entorno de Ejecución	Node.js (v20+ con ES Modules nativos)	Motor de ejecución asíncrono no bloqueante I/O con soporte oficial de modularidad mediante `"type": "module"`.

Componente	Tecnología / Especificación	Propósito Técnico en la Solución
Framework HTTP	Express.js	Manejador de enrutamiento modular, inyección de pipeline de middlewares, parseo de payloads JSON y control de cabeceras.
Persistencia Relacional	PostgreSQL sobre Neon.tech (Cloud Serverless)	Motor relacional conforme a principios ACID, con escalado elástico, llaves foráneas y exigencia estricta de canal cifrado SSL/TLS.
Mapeo Objeto-Relacional	Sequelize ORM (v6+)	Abstracción de modelos relacionales, definición de asociaciones complejas, sanitización automática contra SQL Injection y control de migraciones.
Conectividad de Base de Datos	pg & pg-hstore	Driver nativo de comunicación por sockets TCP y serialización de estructuras entre Node.js y la infraestructura de Neon.tech.
Seguridad Criptográfica	bcrypt	Hashing unidireccional de contraseñas de usuarios con cálculo de costo adaptable (10 rondas de salt) para mitigación de ataques por diccionario.
Control de Acceso / Sesión	JSON Web Tokens (JWT)	Autenticación descentralizada y stateless

Componente	Tecnología / Especificación	Propósito Técnico en la Solución
		(RFC 7519) firmada digitalmente con clave secreta para protección de endpoints privados.
Políticas de Origen Cruzado	CORS (Cross-Origin Resource Sharing)	Habilitación de cabeceras HTTP de seguridad para permitir consumo controlado desde clientes web (React/Next.js), móviles (Flutter) y herramientas API.
Gestión de Configuración	Dotenv	Carga de variables de entorno desacopladas del código fuente en concordancia con los postulados de The Twelve-Factor App.
Control de Versiones	Git & GitHub	Trazabilidad del código fuente, branching y automatización CI/CD con exclusión estricta de secretos y dependencias.
Despliegue Continuo (PaaS)	Render (Cloud Web Service)	Aprovisionamiento automatizado de contenedor web, balanceo de carga, certificados HTTPS gestionados y enlace con GitHub.
Suite de Pruebas	Thunder Client / Postman	Entorno para ejecución y certificación contractual de peticiones HTTP, validación

Componente	Tecnología / Especificación	Propósito Técnico en la Solución
		de payloads, cabeceras y códigos de respuesta.

---

2. Arquitectura de Software, Modularidad y Modelado Relacional

2.1. Arquitectura Modular en Capas

El software se estructuró bajo el patrón arquitectónico Modelo-Controlador-Ruta-Middleware, desacoplando la capa de presentación mediante el intercambio estandarizado de datos en formato JSON:

- src/models/: Define la representación de las entidades de dominio en Sequelize, tipos de datos, restricciones de integridad y asociaciones foráneas.
- **`src/controllers/`**: Implementa la lógica de negocio, validaciones transaccionales y la interacción con los modelos.
- **`src/routes/`**: Expone las rutas y mapea los verbos HTTP hacia las funciones controladoras.
- **`src/middlewares/`**: Intercepta las solicitudes entrantes para verificar la validez del token de sesión antes de autorizar el acceso a recursos protegidos.
- **`src/app.js`**: Instancia Express, configura CORS y analiza cuerpos de petición (`express.json()`).
- **`src/server.js`**: Inyecta variables de entorno e inicializa la escucha del puerto de red.

2.2. Convivencia Técnica entre ES Modules y CommonJS

Para adoptar las normas modernas de JavaScript, se configuró "type": "module" en el package.json. No obstante, la herramienta de control de esquemas sequelize-cli ejecuta nativamente archivos bajo la convención CommonJS (require).

Para solventar esta limitación sin recurrir a transpiladores externos pesados, se diseñó una estrategia híbrida limpia:

1. El código de la aplicación se mantiene 100% en ES Modules (import/export, archivos .js).
2. La configuración del CLI `config/config.cjs` y todas las migraciones en `migrations/*.cjs` utilizan la extensión explícita `.cjs` y sintaxis `module.exports`, permitiendo que `npx sequelize-cli` interprete los archivos como CommonJS mientras la aplicación en tiempo de ejecución opera con ESM.

2.3. Base de Datos en Neon.tech y Conexión Segura SSL

La base de datos relacional opera sobre la infraestructura cloud de Neon.tech. La conexión requiere obligatoriamente canal cifrado TLS/SSL. En src/models/index.js, Sequelize se configuró con:

```javascript
dialectOptions: {
  ssl: {
    require: true,
    rejectUnauthorized: false
  }
}
```

Esto permite establecer el apretón de manos (handshake) TLS seguro contra los certificados dinámicos del clúster cloud.

2.4. Entidades y Diccionario de Datos

Entidad: Clientes (`clientes`)

- id: Entero, autoincremental, clave primaria (INTEGER, primaryKey, autoIncrement).
- nombre: Cadena de caracteres, no nulo (STRING, allowNull: false).
- correo: Cadena de caracteres, no nulo, único, formato de correo válido (STRING, unique: true).
- numLic: Cadena de caracteres, no nulo (STRING, número de licencia de conducción).
- password: Cadena de caracteres, no nulo (STRING, almacena el hash bcrypt de 60 caracteres).

Entidad: Autos (`autos`)

- `id`: Entero, autoincremental, clave primaria (`INTEGER`, `primaryKey`, `autoIncrement`).
- marca: Cadena de caracteres, no nulo (STRING).
- modelo: Cadena de caracteres, no nulo (STRING).
- imagen: Cadena de caracteres (STRING, URL pública de fotografía del vehículo).
- valorAlquiler: Numérico, no nulo (INTEGER/FLOAT, tarifa diaria de renta).
- `anio`: Cadena / Numérico, no nulo.
- `disponibilidad`: Entero (1 = Disponible para alquiler, 0 = Ocupado / En arriendo), valor por defecto: 1.

Entidad: Alquileres (`alquileres`)

- `id`: Entero, autoincremental, clave primaria (`INTEGER`, `primaryKey`, `autoIncrement`).
- `clienteId`: Clave foránea referenciando `clientes(id)`, no nulo.
- `autoId`: Clave foránea referenciando `autos(id)`, no nulo.
- `fechaInicio`: Fecha de inicio del alquiler, no nulo (`DATE`).
- fechaFin: Fecha de culminación del alquiler, no nulo (DATE).
- `estado`: Cadena de texto (`'activo'` o `'finalizado'`), valor por defecto: `'activo'`.

---

3. Seguridad, Autenticación y Autorización

1. **Hashing Criptográfico (`bcrypt`):** Las contraseñas de los clientes son procesadas con salting de 10 rondas antes de persistir en PostgreSQL. La verificación durante el login se realiza mediante comparación temporalmente constante con `bcrypt.compare`.
2. Autenticación sin Estado (JWT): El endpoint /api/clientes/login expide un token firmado con algoritmo HMAC-SHA256 y vigencia temporal (24 horas), codificando el identificador y correo del cliente.
3. Middleware de Intercepción (auth.middleware.js): Valida la presencia de la cabecera Authorization: Bearer <token>, descifra la carga útil mediante jwt.verify e inyecta la identidad del usuario en req.usuario. En

ausencia o inconsistencia del token, responde inmediatamente con código 401 Unauthorized.

4. Protección de Atributos Sensibles: En todas las consultas y serializaciones de respuesta relacionadas con la entidad Cliente, el atributo password se excluye explícitamente (attributes: { exclude: ['password'] }), impidiendo la exposición involuntaria de hashes a clientes de la API.

---

4. Contrato Técnico de Endpoints (Especificación RESTful)

Verbo	Ruta	Acceso	Body / Parámetros	Códigos de Estado	Descripción y Comportamiento
POST	/api/clientes/regi…	Público	JSON: nombre, correo...	201 Created 400 Bad Request 409 Conflict	Crea la cuenta del cliente con password hasheada. Valida duplicidad de correo.
POST	/api/clientes/login	Público	JSON: correo, passwo...	200 OK 400 Bad Request 401 Unauthorized	Verifica credenciales y genera el token de acceso JWT.
GET	/api/clientes/perf…	Protegido	Headers: Authorization: Bearer <token>	200 OK 401 Unauthorized 404 Not Found	Retorna los datos de perfil del usuario autenticado sin password.
POST	/api/autos	Público	JSON: marca, modelo,...	201 Created 400 Bad Request	Añade un nuevo automóvil al catálogo con estado disponible (1).
GET	/api/autos	Público	Ninguno	200 OK	Consulta el catálogo completo de vehículos disponibles.
GET	/api/autos/:id	Público	URL Param: id	200 OK 404 Not Found	Retorna la ficha técnica de un automóvil individual.

Verbo	Ruta	Acceso	Body / Parámetros	Códigos de Estado	Descripción y Comportamiento
POST	/api/alquileres	Protegido	Headers: Authorization: Bearer <token> JSON: autoId, fechaInicio, fechaFin	201 Created 400 Bad Request 401 Unauthorized 404 Not Found	Registra el contrato de alquiler y actualiza la disponibilidad del auto a 0.
GET	/api/alquileres/mi…	Protegido	Headers: Authorization: Bearer <token>	200 OK 401 Unauthorized	Devuelve el historial de alquileres vinculados al cliente autenticado.
PUT	/api/alquileres/:i…	Protegido	Headers: Authorization: Bearer <token> URL Param: id	200 OK 401 Unauthorized 404 Not Found	Concluye el contrato de alquiler (estado finalizado) y libera el auto a disponibilidad 1.

---

5. Control de Versiones y Despliegue Continuo en Render

5.1. Repositorio GitHub y Buenas Prácticas

El código fuente se gestiona bajo control de versiones Git en el repositorio oficial: Repositorio GitHub - api_restful_express.

- **Exclusión de Archivos Sensibles (`.gitignore`):** Se evita la subida de dependencias locales (`node_modules/`), archivos de variables de entorno (`.env`) y registros de log (`*.log`).
- **Documentación de Entorno (`.env.example`):** Se incluye una plantilla base con las claves requeridas (`DATABASE_URL`, `JWT_SECRET`, `PORT`) para garantizar la reproducibilidad del entorno.

5.2. Pipeline de Despliegue Continuo (CI/CD) en Render

El servicio backend se encuentra alojado y operativo en **Render Web Services**:

- **Servicio en Producción:** Servicio en Producción Render - api-restful-express
- **Comando de Compilación (Build Command):** `npm install`
- **Comando de Inicialización (Start Command):** `npm start` (`node src/server.js`)
- **Variables de Entorno Configuradas en Producción:** `DATABASE_URL` (conexión Neon con SSL), `JWT_SECRET` y `PORT`.
- Cada confirmación de cambios (*push*) a la rama `main` de GitHub desencadena una compilación y despliegue automatizado sin caídas del servicio (*zero-downtime deployment*).

6. Registro Técnico Detallado Paso a Paso y Evidencias de Pruebas (Thunder Client)

A continuación se presenta el registro detallado de las pruebas de integración ejecutadas directamente sobre el servicio en producción (`https://api-restful-express.onrender.com`), siguiendo el formato de evidencias técnicas del SENA. Cada prueba cuenta con su enlace directo a la captura correspondiente en la nube.

La totalidad de las capturas se encuentra organizada en la Carpeta de Evidencias en Google Drive - ExpressJS_Evidencias.

Paso 1 — Inspección Inicial del Catálogo de Autos en Producción

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Navegador Web / Cliente HTTP apuntando a la URL pública de producción
Acción Técnica:	Verificación de disponibilidad del servicio y consulta pública del inventario de autos
Método y URL:	`GET https://api-restful-express.onrender.com/api/autos`

Parámetros / Headers:	Ninguno (Acceso público)
Código HTTP / Respuesta:	`200 OK` | Retorna array con vehículos registrados (ej. Toyota Corolla, ID: 1, disponibilidad: 1)
Explicación Técnica:	Comprueba que el Web Service en Render y la base de datos en Neon.tech se encuentran comunicados correctamente mediante SSL y que los endpoints públicos responden con estructura JSON homogénea.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 1** *Enlace directo: https://drive.google.com/file/d/1c2QogX6nLuJNxsJY2EisVtfdsbIfCWpk/view?usp=drivesdk*

Paso 2 — Panel de Control y Colección de Pruebas en Thunder Client

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client Extension (Visual Studio Code) — Panel de Actividad
Acción Técnica:	Organización de la suite de pruebas de integración y trazabilidad de peticiones
Método y URL:	Entorno Thunder Client (Gestión de historial local y llamadas a Render)
Parámetros / Headers:	Historial de peticiones dirigidas a `https://api-restful-express.onrender.com`
Código HTTP / Respuesta:	Monitoreo general de estados HTTP (`200 OK`, `201 Created`, `401 Unauthorized`, `409 Conflict`)
Explicación Técnica:	Establece la plataforma de pruebas para validar contractualmente el comportamiento de cada endpoint frente al backend desplegado en la nube.

Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 2** *Enlace directo: https://drive.google.com/file/d/1xxUPhi7_cDXQhfanwQ_I6-bEr1A-LdUh/view?usp=drivesdk*

Paso 3 — Prueba 1: Registro Exitoso de un Nuevo Cliente

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña de Registro de Clientes
Acción Técnica:	Envío de solicitud de creación de cuenta de usuario con contraseña en texto plano para hashing
Método y URL:	`POST https://api-restful-express.onrender.com/api/clientes/registro`
Cuerpo de la Petición:	`{"nombre": "Carlos Perez", "correo": "carlos.perez@ejemplo.com", "numLic": "LIC-9988", "password": "claveSegura123"}`
Código HTTP / Respuesta:	`201 Created` | `{"id": 2, "nombre": "Carlos Perez", "correo": "carlos.perez@ejemplo.com"}` (Tiempo: ~1.67 s)
Explicación Técnica:	El controlador intercepta los datos de registro, aplica el algoritmo `bcrypt.hash` sobre la contraseña con 10 rondas de salt y persiste la entidad en PostgreSQL. El campo `password` es omitido de la respuesta para salvaguardar la privacidad.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 3** *Enlace directo: https://drive.google.com/file/d/1RXaCz3udX6sAEVOEWdU7bxt0ZM3IYcYL/view?usp=drivesdk*

Paso 4 — Prueba 2: Autenticación y Generación de Token JWT

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña de Login
Acción Técnica:	Envío de credenciales de acceso para verificación criptográfica y emisión de token de sesión
Método y URL:	`POST https://api-restful-express.onrender.com/api/clientes/login`
Cuerpo de la Petición:	`{"correo": "carlos.perez@ejemplo.com", "password": "claveSegura123"}`
Código HTTP / Respuesta:	`200 OK` | Retorna objeto JSON con mensaje confirmatorio y string del `token` JWT firmado digitalmente
Explicación Técnica:	El backend busca al cliente por correo electrónico, valida la contraseña utilizando `bcrypt.compare` y, al comprobar su validez, emite un JSON Web Token firmado con la clave secreta institucional y vigencia de 24 horas.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 4** *Enlace directo: https://drive.google.com/file/d/16FMg-uJt3tGQJuZaf09V5j3TJdGrqS5P/view?usp=drivesdk*

Paso 5 — Prueba 3: Creación de Nuevo Vehículo en el Catálogo

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña de Catálogo de Autos
Acción Técnica:	Aprovisionamiento de un nuevo vehículo dentro de la flota operativa

Método y URL:	`POST https://api-restful-express.onrender.com/api/autos`
Cuerpo de la Petición:	`{"marca": "Mazda", "modelo": "CX-30", "imagen": "https://example.com/mazda.jpg", "valorAlquiler": 150000, "anio": "2024", "disponibilidad": 1}`
Código HTTP / Respuesta:	`201 Created` | `{"id": 2, "marca": "Mazda", "modelo": "CX-30", "valorAlquiler": 150000, "disponibilidad": 1, ...}` (Tiempo: 785 ms)
Explicación Técnica:	Se valida la inserción de registros en la tabla `autos`. La propiedad `disponibilidad: 1` indica que el vehículo ingresa inmediatamente como elegible para ser rentado por cualquier cliente registrado.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 5** *Enlace directo: https://drive.google.com/file/d/1wIgxi5NoBlsDR7RxP2LlatOhNZYZuou3/view?usp=drivesdk*

Paso 6 — Inyección de Autorización Bearer Token en Thunder Client

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña Auth (Bearer Token)
Acción Técnica:	Configuración de credenciales de autorización para el consumo de endpoints protegidos
Método y URL:	Configuración de cabeceras previas para `/api/alquileres`

Parámetros / Headers:	Tipo: `Bearer` | Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiY29ycmVvIjoiY2FybG9z...`
Código HTTP / Respuesta:	Parámetro de autenticación almacenado para inyección automática en cabecera `Authorization`
Explicación Técnica:	Demuestra el cumplimiento del protocolo de seguridad donde las rutas de alquiler exigen que el cliente adjunte el token obtenido en la autenticación previa para validar su identidad sin almacenar sesiones en el servidor.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 6** *Enlace directo: https://drive.google.com/file/d/15PlI930NsqTX5UVK2wM-DbbFqVcTJZPa/view?usp=drivesdk*

Paso 7 — Prueba 5: Reserva de Alquiler y Cambio de Disponibilidad del Auto

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña de Registro de Alquiler
Acción Técnica:	Apertura de contrato de alquiler y actualización transaccional de inventario
Método y URL:	`POST https://api-restful-express.onrender.com/api/alquileres`
Headers y Body:	Headers: `Authorization: Bearer <token>` | Body: `{"autoId": 1, "fechaInicio": "2026-10-01", "fechaFin": "2026-10-05"}`
Código HTTP / Respuesta:	`201 Created` | `{"mensaje": "Alquiler registrado exitosamente", "alquiler": {"id": 2,`

	`"clienteId": 2, "autoId": 1, "estado": "activo", ...}}`
Explicación Técnica:	El middleware valida el token y extrae `clienteId: 2`. El controlador crea el alquiler en estado `'activo'` e invoca la actualización del vehículo (`Auto.update({ disponibilidad: 0 })`), evitando que el auto pueda ser reservado por otro usuario simultáneamente.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 7** *Enlace directo: https://drive.google.com/file/d/1cpB9iQfR1oQl3YRfOfqWuZDgqp9tPEtz/view?usp=drivesdk*

Paso 8 — Prueba 6: Consulta del Historial de Alquileres del Cliente

Parámetro Técnico	**Especificación / Valor Evaluado**
Pantalla / Interfaz:	Thunder Client — Pestaña Historial Personal de Alquileres
Acción Técnica:	Consulta protegida de los contratos de alquiler pertenecientes al usuario autenticado
Método y URL:	`GET https://api-restful-express.onrender.com/api/alquileres/mis-alquileres`
Headers:	`Authorization: Bearer <Token_JWT_Carlos_Perez>`
Código HTTP / Respuesta:	`200 OK | [{"id": 2, "fechaInicio": "2026-10-01...", "autoId": 1, "estado": "activo", "Auto": {"marca": "Toyota", "modelo": "Corolla", "valorAlquiler": 120000}}]`
Explicación Técnica:	El endpoint recupera los alquileres filtrados estrictamente por el `clienteId` contenido en el token, realizando un

	JOIN relacional con la tabla `autos` para retornar de forma embebida los detalles del vehículo arrendado.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 8** *Enlace directo: https://drive.google.com/file/d/1e_TqE6PLiAVMqnNYZtWXE9IrfwBiBbAz/view?usp=drivesdk*

Paso 9 — Prueba 7: Finalización del Alquiler y Liberación del Vehículo

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Pestaña de Cierre de Alquiler
Acción Técnica:	Devolución del vehículo, cambio de estado de contrato y restauración de disponibilidad
Método y URL:	`PUT https://api-restful-express.onrender.com/api/alquileres/2/finalizar`
Headers y Parámetros:	Headers: `Authorization: Bearer <token>` | URL Param: `id: 2`
Código HTTP / Respuesta:	`200 OK` | `{"mensaje": "Alquiler finalizado exitosamente"}` (Tiempo: 1.44 s)
Explicación Técnica:	El controlador localiza el alquiler especificado, comprueba que pertenezca al usuario o posea permisos adecuados, muta el campo `estado` a `'finalizado'` y revierte el indicador del auto asociado (`disponibilidad: 1`), reintegrándolo al inventario público.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 9** *Enlace directo: https://drive.google.com/file/d/1qpR9bhFRfUIltjeZX5Vs2NusslHakd37/view?usp=drivesdk*

Paso 10 — Prueba 8a: Excepción de Seguridad por Ausencia de Token (401 Unauthorized)

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Prueba de Rechazo sin Autenticación
Acción Técnica:	Intento de acceso a ruta privada omitiendo la cabecera `Authorization`
Método y URL:	`GET https://api-restful-express.onrender.com/api/alquileres/mis-alquileres`
Headers:	Ninguno (Sin cabecera de autorización)
Código HTTP / Respuesta:	`401 Unauthorized` | `{"mensaje": "Token no proporcionado"}` (Tiempo: 261 ms)
Explicación Técnica:	Comprueba la efectividad perimetral del middleware `auth.middleware.js`, el cual rechaza la petición de forma inmediata antes de ejecutar cualquier consulta a la base de datos, protegiendo los recursos privados.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 10** *Enlace directo: https://drive.google.com/file/d/1zAuOue8ao6fzQh2NFuUITv8L4IJz9LM5/view?usp=drivesdk*

Paso 11 — Prueba 8b: Excepción por Conflicto de Duplicidad de Correo (409 Conflict)

Parámetro Técnico	Especificación / Valor Evaluado
Pantalla / Interfaz:	Thunder Client — Prueba de Validación de Integridad de Datos
Acción Técnica:	Intento de registro con una dirección de correo electrónico ya existente en el sistema

Método y URL:	`POST https://api-restful-express.onrender.com/api/clientes/registro`
Cuerpo de la Petición:	`{"nombre": "Carlos Perez", "correo": "carlos.perez@ejemplo.com", "numLic": "LIC-9988", "password": "claveSegura123"}`
Código HTTP / Respuesta:	`409 Conflict` | `{"mensaje": "Ese correo ya esta registrado"}` (Tiempo: 1.47 s)
Explicación Técnica:	El controlador evalúa la existencia previa del correo mediante `Cliente.findOne({ where: { correo } })`. Al constatar colisión, interrumpe el flujo y responde con el código semántico `409 Conflict`, preservando la restricción de unicidad de la base de datos.
Evidencia Fotográfica:	🔍 **Ver Captura en Google Drive - Evidencia 11** *Enlace directo: https://drive.google.com/file/d/1vuBa-P9nv-_EBh1RMYndTcfcdouolbrw/view?usp=drivesdk*

---

7. Checklist de Cumplimiento Técnico (Sección 18 Guía SENA)

Numeral	Criterio de Aceptación / Requerimiento Técnico	Estado	Verificación Técnica
01	Inicialización del proyecto con soporte ES Modules (`"type": "module"`)	**Cumplido**	Habilitado en `package.json`, utilizando `import` y `export` nativos.

02	Servidor Express desacoplado con modularidad de middlewares	**Cumplido**	Separación entre `app.js` (pipeline) y `server.js` (puerto).
03	Configuración de variables de entorno con `dotenv` y `.env.example`	**Cumplido**	Variables sensibles protegidas e ignoradas en Git.
04	Base de datos PostgreSQL en Neon.tech con conexión cifrada SSL	**Cumplido**	Conexión cloud con `ssl: { require: true, rejectUnauthorized: false }`.
05	Configuración híbrida de Sequelize para migraciones en `.cjs`	**Cumplido**	Compatibilidad CLI resuelta mediante archivos `.cjs`.
06	Definición de modelos relacionales (`Cliente`, `Auto`, `Alquiler`)	**Cumplido**	Modelos mapeados con tipos de datos, restricciones y validaciones.
07	Relaciones y asociaciones relacionales (1:N)	**Cumplido**	Asociaciones `hasMany` y `belongsTo` con carga ansiosa (`include`).
08	Encriptación unidireccional de credenciales con `bcrypt`	**Cumplido**	Factor de costo de 10 rondas; contraseñas nunca almacenadas en plano.
09	Autenticación stateless basada en JSON Web Tokens (JWT)	**Cumplido**	Firma de tokens en login y validación estricta en middleware.

10	Exclusión de atributos confidenciales en respuestas JSON	**Cumplido**	Respuestas de clientes sin inclusión del hash `password`.
11	Operaciones de catálogo e inventario vehicular	**Cumplido**	Creación y listado de vehículos funcionando correctamente.
12	Lógica de negocio de alquiler y control de disponibilidad	**Cumplido**	Modificación de disponibilidad de auto (`1 -> 0`) y reversión (`0 -> 1`).
13	Manejo semántico de respuestas HTTP y excepciones	**Cumplido**	Códigos `200`, `201`, `400`, `401`, `404` y `409` implementados.
14	Control de versiones y buenas prácticas en GitHub	**Cumplido**	Repositorio público limpio, con `.gitignore` exhaustivo.
15	Despliegue continuo en la nube (Render Web Services)	**Cumplido**	Servicio web publicado y enlazado con la rama `main`.
16	Certificación de pruebas de integración con Thunder Client	**Cumplido**	8 escenarios validados con 11 evidencias fotográficas completas.

---

8. Conclusiones y Recomendaciones Técnicas

5. **1. Gestión de la Modularidad en el Ecosistema Node.js:** La transición a ECMAScript Modules (`"type": "module"`) representa el estándar contemporáneo de desarrollo en JavaScript. La estrategia de adoptar archivos `.cjs` para las herramientas del CLI de Sequelize demostró ser

una solución arquitectónica limpia que garantiza compatibilidad sin comprometer la modernidad del código fuente.

6. **2. Resiliencia de Conexiones Cloud Serverless:** La implementación de bases de datos serverless como Neon.tech exige contemplar políticas específicas de negociación TLS/SSL. Configurar apropiadamente el dialecto de Sequelize permitió una comunicación segura y tolerante frente a las reconexiones propias de arquitecturas serverless.
7. **3. Seguridad Stateless mediante JWT:** El desacoplamiento logrado mediante JSON Web Tokens permitió proteger endpoints críticos sin sobrecargar la infraestructura con sesiones en memoria o consultas recurrentes a base de datos para autenticación, facilitando la escalabilidad horizontal del servicio.
8. **4. Recomendaciones Futuras:** Se recomienda implementar una capa de validación de esquemas basada en bibliotecas como `Zod` o `Joi` para sanitizar las entradas antes de llegar al controlador, así como encapsular las operaciones de reserva y devolución bajo transacciones administradas (`sequelize.transaction`) para asegurar consistencia transaccional absoluta ante eventuales fallos de red.

---

9. Enlaces y Anexos

- **Repositorio de Código Fuente (GitHub):** Repositorio GitHub - api_restful_express
- **Servicio Desplegado en Producción (Render):** Servicio en Producción Render - api-restful-express
- **Carpeta Central de Evidencias en Google Drive:** Carpeta de Evidencias en Google Drive - ExpressJS_Evidencias
- **Inventario de Capturas de Pantalla:**
    - Captura de Evidencia 1 — Consulta pública de catálogo en Render
    - Captura de Evidencia 2 — Suite y panel de actividad en Thunder Client
    - Captura de Evidencia 3 — Prueba 1: Registro de nuevo cliente (201 Created)
    - Captura de Evidencia 4 — Prueba 2: Autenticación y obtención de Token JWT (200 OK)

- Captura de Evidencia 5 — Prueba 3: Creación de vehículo en catálogo (201 Created)
- Captura de Evidencia 6 — Inyección de Bearer Token en Thunder Client
- Captura de Evidencia 7 — Prueba 5: Registro de alquiler exitoso (201 Created)
- Captura de Evidencia 8 — Prueba 6: Historial de alquileres del cliente con asociación (200 OK)
- Captura de Evidencia 9 — Prueba 7: Finalización de alquiler y liberación del auto (200 OK)
- Captura de Evidencia 10 — Prueba 8a: Excepción 401 Unauthorized sin Token
- Captura de Evidencia 11 — Prueba 8b: Excepción 409 Conflict por duplicidad de correo